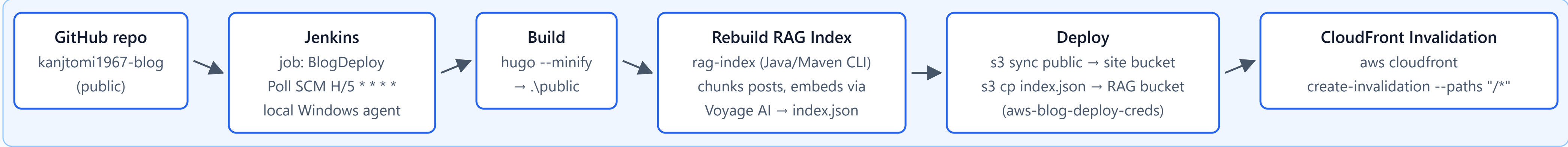


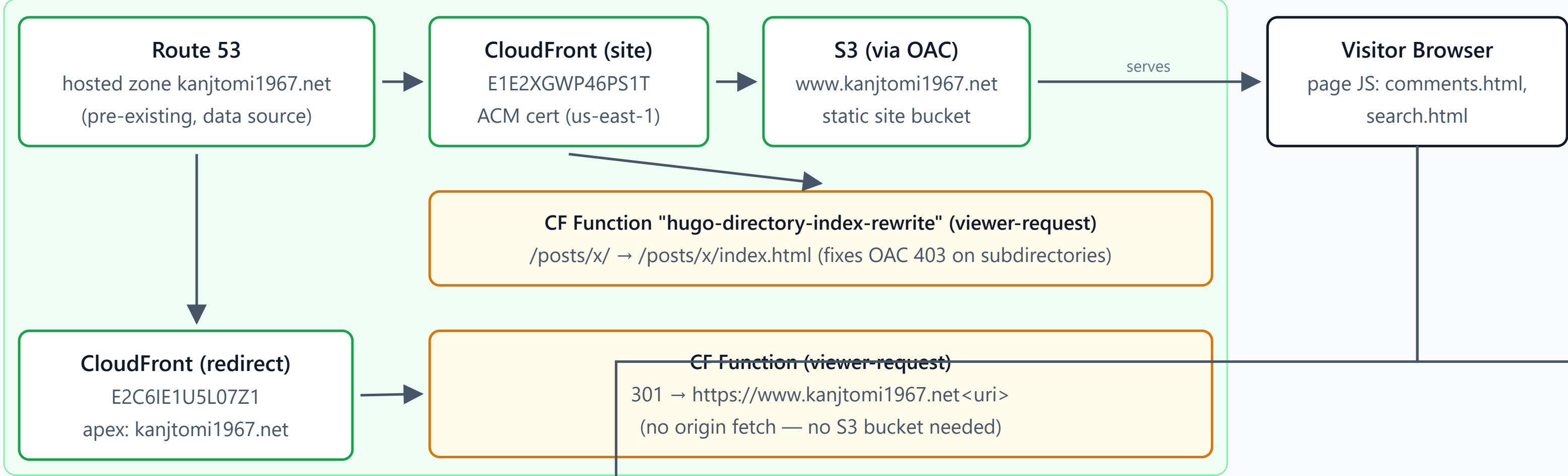
www.kanjtomi1967.net — Blog Architecture

Static Hugo site on AWS (S3 + CloudFront), self-hosted comments and RAG search via Lambda

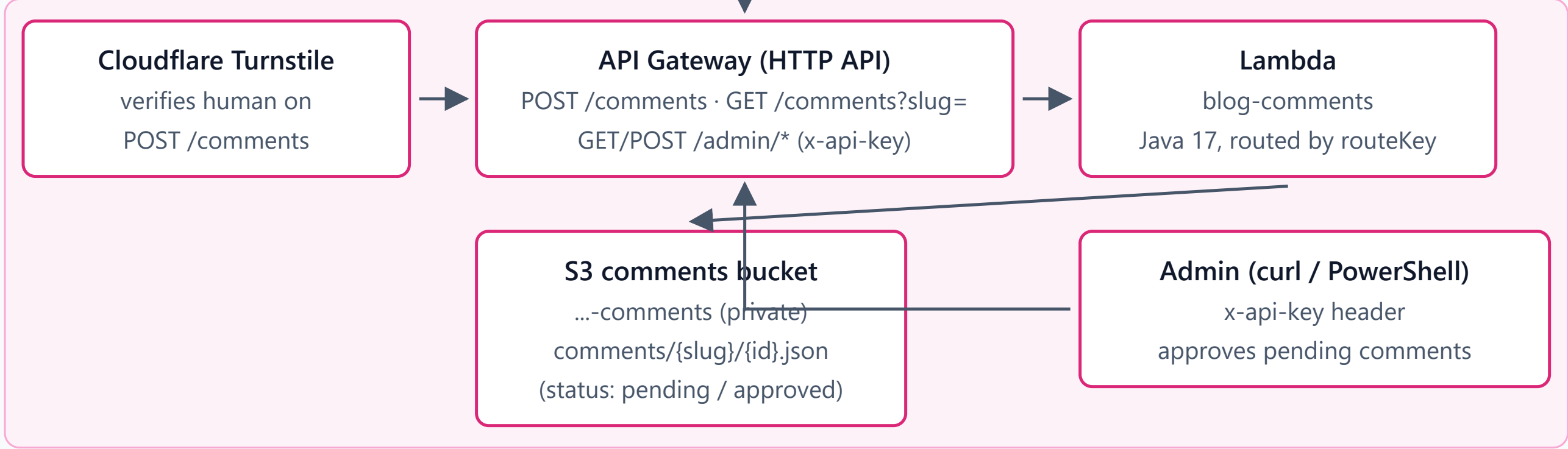
CI/CD Pipeline (local Jenkins on Windows)



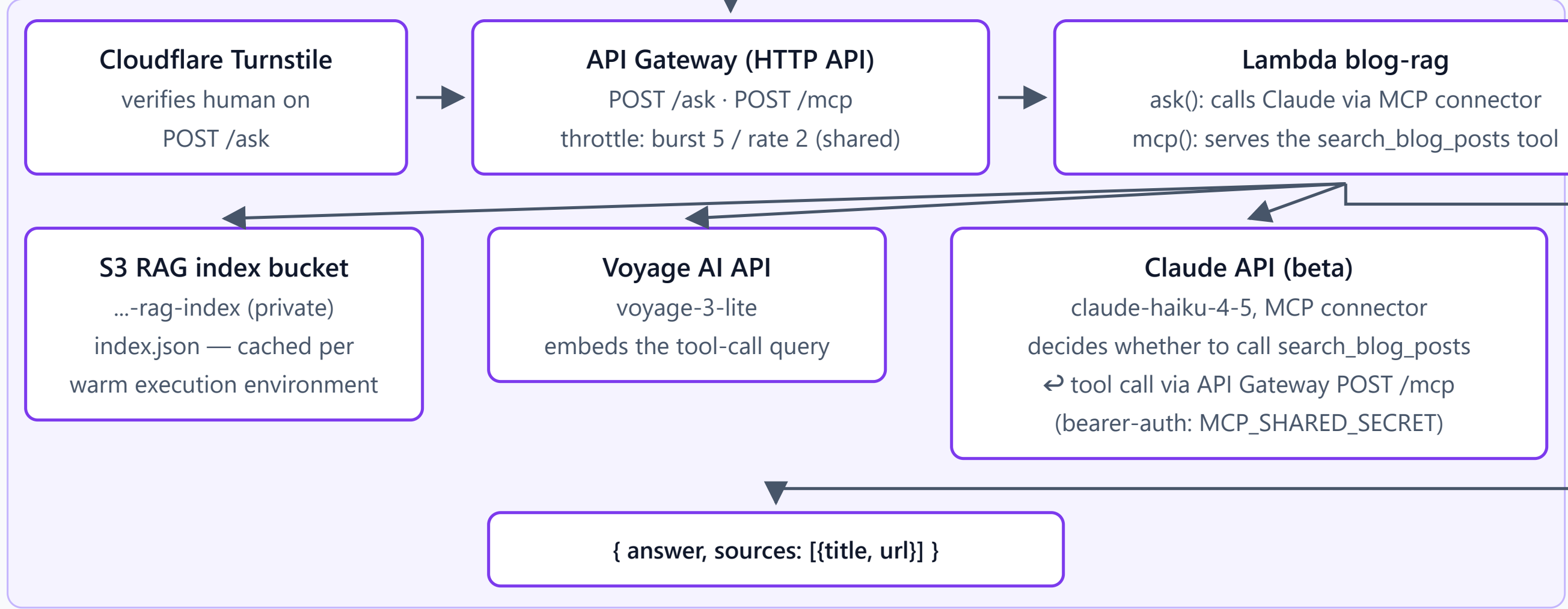
DNS, CDN & Static Hosting



Comment System (Lambda + S3, moderated)



RAG Search — "AI に聞く" (Lambda + Voyage + Claude)



Notes

Estimated cost: ~\$1-3/month (Route 53 zone + small redirect distribution); Lambda/API Gateway usage stays within/near free tier for personal traffic.
Secrets (Turnstile secret, admin_api_key, Voyage/Anthropic API keys, mcp_shared_secret) live only in terraform.tfvars (gitignored) and Lambda env vars — never committed.
Out of scope: no user login/accounts, no likes/reactions. Comments are anonymous + name field, moderated via x-api-key admin routes.
Region: ap-northeast-1 for all resources except ACM certificates, which must be requested in us-east-1 for CloudFront.

モジュール解説

CI/CD パイプライン(ローカル Jenkins on Windows)

GitHub repo

ソースを管理する公開リポジトリ(kanjtomi1967-blog)。Markdown記事、Hugo設定、Terraform、Lambdaコードなど、サイトを構成する全てをここで管理する。

Jenkins

ローカルWindows上で稼働する自前のCI/CDサーバー(ジョブ名 BlogDeploy)。GitHubから直接Webhookを受けられないため、5分間隔のPoll SCM(H/5 * * * *)で新しいコミットを検知してビルドを起動する。

Build

hugo --minify を実行し、Markdown記事を静的HTMLへビルドして .¥public に出力するステージ。

Rebuild RAG Index

rag-index(Java/Maven製CLI)が全記事を読み込み、見出し単位でチャンク分割してVoyage AIで埋め込みベクトル化し、index.jsonを生成する。デプロイのたびに実行され、常に最新記事を検索対象に反映する。

Deploy

ビルド済みサイトをS3の本番バケットへ `aws s3 sync`、RAGインデックスを専用S3バケットへ `aws s3 cp` でアップロードするステージ。Jenkins Credentials(aws-blog-deploy-creds)のIAM権限を使用する。

CloudFront Invalidation

デプロイ内容を即座に反映させるため、`aws cloudfront create-invalidation --paths "/*"` でCDNキャッシュを全パス無効化する。

DNS・CDN・静的ホスティング

Route 53

ドメイン kanjtomi1967.net の既存ホストゾーン。Terraformではデータソースとして参照するのみで、ゾーン自体は管理対象外(既存インフラ)。

CloudFront(site)

www.kanjtomi1967.net 向けの本番配信ディストリビューション。オリジンはS3(OAC経由)。CloudFront向けの制約により、TLS証明書は us-east-1 で発行したACM証明書を使用する。

S3(via OAC)

ビルド済み静的サイトを格納する非公開バケット。パブリックアクセスは一切許可せず、CloudFrontのOrigin Access Control経由でのみ読み取り可能。

CF Function "hugo-directory-index-rewrite"

/posts/x/ のようなディレクトリ形式のURIを /posts/x/index.html へ、viewer-request時に書き換えるエッジ関数。OAC経由のS3はルート以外のディレクトリindexを自動解決しないため、これがないと403 AccessDeniedになる。

CloudFront(redirect)

apexドメイン(kanjtomi1967.net)専用の、トラフィックがほぼ発生しない軽量の2つ目のディストリビューション。

CF Function(apex redirect)

全リクエストを https://www.kanjtomi1967.net<uri> へ301リダイレクトするエッジ関数。オリジンへのフェッチが一切発生しないため、apex用の公開S3バケットが不要(アカウントのS3 Block Public Accessとも衝突しない)。

Visitor Browser

実際にサイトを閲覧するユーザーのブラウザ。コメント欄・RAG検索のJS(comments.html / search.html)を実行し、それぞれのAPIを呼び出す。

コメントシステム(Lambda + S3、承認制)

Cloudflare Turnstile

コメント投稿(POST /comments)がbotではなく人間によるものかをサーバー側で検証する、CAPTCHA代替サービス。

API Gateway(HTTP API)

コメント関連の全エンドポイントをまとめるルーター。公開ルート(POST /comments、GET /comments?slug=)と、x-api-key必須の管理者ルート(/admin/*)を持つ。

Lambda blog-comments

Java 17製の単一Lambda。API GatewayのrouteKeyで内部的にルーティングし、投稿・取得・承認処理をすべて1つの関数で担う。

S3 comments bucket

非公開バケット(...-comments)。コメント1件ごとに comments/{slug}/{id}.json として保存し、pending / approved のstatusフィールドで管理する。

Admin(curl / PowerShell)

専用の管理UIは持たず、x-api-keyヘッダー付きのリクエストでpending中のコメントを一覧・承認する運用(scripts/comments-admin.md参照)。

RAG検索 — 「AI に聞く」 (Lambda + Voyage + Claude、MCPコネクタ経由)

Cloudflare Turnstile

質問送信(POST /ask)側でも同様にbot対策として検証を行う。コメント側とは別実装だが同じ仕組みを再利用している。

API Gateway(HTTP API)

POST /ask(ブラウザ向け)と POST /mcp(Anthropicのバックエンドがコールバックする、MCPサーバーとしてのルート)の2ルートを持つ。Claude API呼び出しコストが暴走しないよう、バースト5・レート2の保守的なスロットリングを両ルートで共有している。

Lambda blog-rag

Java 17製、1つのLambdaが2つの役割を持つ。ask()は質問を受け取ると、記事をローカルで検索せず、Claude APIをMCPコネクタ(beta mcp-client-2025-11-20)付きで呼び出す。mcp()はステートレスなMCP Streamable HTTPサーバーで、search_blog_postsツールを1つだけ公開する — Voyage AIでの埋め込み、S3インデックスに対するコサイン類似度の総当たりランキング(上位4チャンク)は、このmcp()側に移動した。

S3 RAG index bucket

非公開バケット(...-rag-index)。rag-index CLIが生成した index.json(記事チャンクとその埋め込みベクトル式)を保管する。mcp()がsearch_blog_postsツール呼び出しのたびに参照する。

Voyage AI API

voyage-3-liteモデルで、mcp()がsearch_blog_postsツール呼び出しの引数(query)を埋め込みベクトルに変換する。

Claude API(beta)

claude-haiku-4-5。ask()から質問文だけを渡され、必要に応じてsearch_blog_postsツールを自律的に呼び出す(Lambdaが上位4チャンクを事前に取得してプロンプトに詰め込む方式はやめた)。ツール呼び出しはAnthropicのバックエンドから直接、このAPI Gatewayの POST /mcp へ、Bearerトークン(MCP_SHARED_SECRET)付きで行われる。ブログの内容でカバーされていない質問には、外部知識で答えず正直にその旨を伝えるよう指示されている。

sources の抽出

MCPコネクタのレスポンスには通常のテキストに加え、mcp_tool_resultブロック(ツールが実際に返した内容)が含まれる。ask()はそのテキストから ### タイトル / Source: URL のペアを正規表現で拾い、URL単位で重複排除して sources を組み立てる。

レスポンス

{ answer, sources: [{title, url}] } 形式でフロントエンドに返却される(契約はMCP化前と同じ)。

備考

- 想定コスト: 月額 約\$1〜3(Route 53ホストゾーン + 小規模なりダイレクト用CloudFront)。
CloudFront/S3/Lambda/API Gatewayの利用量は、個人ブログ規模のトラフィックであれば無料利用枠内〜ほぼ無料枠に収まる想定。

- シークレット管理: Turnstileのシークレットキー、admin_api_key、Voyage/Anthropic APIキー、mcp_shared_secret(POST /mcpの認証用)は terraform.tfvars(gitignore対象)とLambdaの環境変数にのみ存在し、リポジトリにはコミットされない。
- スcope外: ユーザーログイン/アカウント機能、いいね/リアクション機能は実装しない。コメントは匿名+名前欄のみで、承認制で運用する。
- リージョン: ACM証明書(CloudFront向け)のみ us-east-1、それ以外の全リソースは ap-northeast-1。

付録: MCP(Model Context Protocol)

このリポジトリには、性質の異なる2つのMCP関連コンポーネントがある。ひとつは**本番のRAG検索が実際に使っているMCP接続**(blog-rag Lambda)、もうひとつは**学習用の独立したサンプル** (mcp-blog-sample/)。両者は無関係で、コードも通信経路も共有しない。

MCP とは

Model Context Protocol は、LLMクライアント(Claude API、Claude Desktop、Claude Codeなど)が外部の ツールやデータソースに統一的な方法でアクセスするための、Anthropicが策定したオープンな規格。サーバー側が「ツール」「リソース」「プロンプト」を公開し、クライアント側がそれを検出・呼び出す。標準入出力(stdio)やHTTPなどのトランスポート上でJSON-RPCメッセージをやり取りする。

本番: RAG検索のMCPコネクタ統合(blog-rag)

「RAG検索」セクション(2ページ目参照)で説明した通り、blog-rag Lambdaの POST /ask は、もはや自分でVoyage AI/S3を呼んで上位チャンクをプロンプトに詰め込まない。代わりにAnthropic APIの**MCPコネクタ**(beta、ヘッダー mcp-client-2025-11-20)を使い、質問文だけをClaudeに渡す。Claudeは必要と判断した場合にだけ、自律的に search_blog_posts ツールを呼び出す。

接続方式

Javaの com.anthropic:anthropic-java SDK(2.57.0 に更新)で client.beta().messages().create(...) を呼び、 mcp_servers (接続先URLとBearerトークン) と tools (mcp_toolset) を指定する。実行そのものはAnthropicのバックエンドが行う — このLambdaが自分でツール呼び出しループを 回すわけではない。

ツールを提供する側

同じ blog-rag Lambdaが、新設の POST /mcp ルートでもう一つの顔を持つ。Anthropicのバックエンドがこのエンドポイントに直接コールバックし、ステートレスなMCP Streamable HTTP サーバーとして initialize / tools/list / tools/call のJSON-RPCに応答する。tools/call で初めてVoyage AIによる埋め込みと、S3インデックスに対するコサイン類似度検索(上位4チャンク)が実行される — これは以前 ask() が毎回無条件に行っていた処理そのもので、場所が移っただけ。

認証

POST /mcp は MCP_SHARED_SECRET (Terraform変数 mcp_shared_secret)によるBearer認証で保護されている。Claudeへのリクエスト側で authorization_token として同じ値を渡すことで、Anthropicのバックエンドだけがこのツールを 呼び出せるようにしている。

sourcesの取り出し方

MCPコネクタ経由のレスポンスには、通常のテキストブロックに加えて `mcp_tool_result` ブロック(ツールが実際に返した内容)が含まれる。`ask()` はこの中の `### タイトル / Source: URL` という(このコード自身が決めた)書式を正規表現で拾い、フロントエンド向けの `sources` を組み立てる。

学習用サンプル: `mcp-blog-sample/`

上記の本番統合とは無関係の、MCP学習用の独立したコンポーネント。本番のHugoビルド・Jenkinsパイプラインには組み込まれておらず、AWSインフラも一切使わない。

このリポジトリでの実装

配置場所

`mcp-blog-sample/` (独立したPythonプロジェクト。本番のHugoビルド・Jenkinsパイプラインとは無関係)

実装言語 / SDK

Python。`mcp` パッケージ(バージョン2.0系)の `MCPServer` クラスを使用(2.0系での名称。旧バージョンでは `FastMCP` という名前だった)。デコレータ `@mcp.tool()` を関数に付けるだけでツールとして公開でき、`docstring`がそのままツールの説明、型ヒントから入力スキーマが自動生成される。

データソース

`content/posts/*.md` (ブログ記事のMarkdownファイル)をその場で読み込み、YAMLフロントマター (`title/date/tags/draft`)と本文を都度パースする。ビルド成果物やデータベースには依存しない。

公開しているツール

`list_posts(include_drafts=False)`

全記事の`title/date/tags/slug/draft`を一覧で返す(本文は含まない)。`include_drafts=True` を指定しない限り下書き記事は除外される。

`get_post(slug)`

指定`slug`の記事1件について、本文(`body`)を含む全データを返す。該当記事がなければエラーを返す。

`search_by_tag(tag)`

指定タグを持つ非下書き記事のみを検索して一覧で返す。

起動・通信方式

`mcp.run()` はデフォルトで**stdioトランスポート**を使う。つまりクライアント (Claude Desktopなど) がこのPythonスクリプトを子プロセスとして起動し、標準入出力でJSON-RPC メッセージ(ツール一覧の問い合わせ、ツール呼び出し、結果の返却)をやり取りする。ネットワークポートは開かず、常駐サーバーでもない — クライアントが必要な時だけ起動し、不要になれば終了する。

動作確認の方法

- **MCP Inspector**(`mcp dev server.py`) — ブラウザのUIからツールを対話的に呼び出して 確認できる開発者向けツール。実行するとInspector Web(<http://127.0.0.1:6274>)とサンドボックス (:6275)が起動し、ブラウザが自動で開く。内部的にnpx経由でNode.js製のInspectorを併用する。
- **実クライアントによる接続テスト** — `mcp` パッケージの `ClientSession` + `stdio_client` を使ったPythonスクリプトで、実際のMCPプロトコル(初期化 → `list_tools` → `call_tool`)を通して検証済み。3ツールすべてが正しく検出され、期待通りのJSONを返すことを確認した。

Claude Desktop への接続

`claude_desktop_config.json` の `mcpServers` に、venv内の `python.exe` と `server.py` の絶対パスを登録することで、Claude Desktop本体からこの3ツールを直接呼び出せるようになる(`mcp-blog-sample/README.md` に設定例あり)。この接続は 上記の本番RAG統合とは完全に独立しており、互いに影響しない。

参考: この2つのMCP統合とは別に、この会話を行っているClaude Code自体も、ブラウザ操作用の `claude-in-chrome` や、Gmail / Google Calendar / Google Drive等と連携するMCPサーバーに 接続されている。これらはClaude Codeの実行環境側の設定であり、本リポジトリのコードとは無関係。